

二维数组的"方寸之间" (17分)

```
1  #include <stdio.h>
2  int main() {
3      int a[2][3] = {{1,2,3},
4                      {4,5,6}};
5      int *p = a[0];
6      int (*q)[3] = a;
7
8      printf("%d\n", *(p + 3) + (*(q + 1) + 1));
9      printf("%d\n", *(a[1] - 1) + *(a[0] + 4));
10     printf("%zu\n", sizeof(a + 1) + sizeof(*a));
11     printf("%d\n", (p + 2)[1] - (*(q + 1))[-1]);
12     return 0;
13 }
```

问题

1. 写出程序输出结果 (4分)

2. 逐个表达式分析

- `*(p + 3)` 访问的是哪个元素? 为什么? (2分)
 - 访问的是 `a[1][0]` (即4)
 - 原因: `p` 指向 `a[0][0]`, 指针+1就移动一个 `int` 大小, +3偏移到第4个元素, 在连续存储中即第1行第0列。
- `*(a[1] - 1)` 的值是多少?数组在内存中存储有什么特点? (3分)
 - `a[1]` 指向元素 4, 减1后指向前一元素 3, 因为二维数组在内存中是连续线性存储的
- `sizeof(a + 1)` 的结果是多少? 如果是 `sizeof(a)` 值又是多少? 说明原因 (2分)
 - 在表达式中, `a` 退化为指针, `a+1` 是指针类型 (`int (*)[3]`), 其大小固定为指针大小 (64位系统是 8, 32位系统是 4)
 - `sizeof(a)` 结果: 24 ($2 * 3 * \text{sizeof}(\text{int}) = 2 * 3 * 4 = 24$)
在 `sizeof(a)` 中: `a` 是数组名, 但 `sizeof` 是特殊情况, 不会发生退化, `sizeof` 直接计算整个数组所占的字节数, 数组 `a[2][3]` 有 2 行, 每行 3 个 `int`, 共 6 个 `int`, 每个 `int` 占 4 字节, 结果为 24
- `(p + 2)[1]` 等价于什么形式的指针运算? (1分)
 - 等价于 `*(p + 2 + 1)`, 即 `*(p + 3)`
- `*(q + 1))[-1]` 中的 `[-1]` 访问的是哪个元素? 并说明原因 (2分)
 - 访问的是 `a[0][2]`, 即 3
 - 因为 `*(q+1)` 是第1行首地址, `[-1]` 表示向前一个元素, 即第0行最后一个元素。

3. 概念辨析

- `p` 和 `q` 的类型分别是什么？它们的步长各是多少？ (2分)
 - `p` 类型：数组指针，`int *`，步长 = `sizeof(int)` = 4字节
 - `q` 类型：指针数组，`int (*)[3]`（指向含3个int的数组的指针），步长 = `3 * sizeof(int)` = 12字节
- 如果数组改为 `int a[2][3] = {0};`，`sizeof(a+1) + sizeof(*a)` 的结果会变吗？为什么？ (2分)
 - 不会变
 - 因为 `sizeof` 在编译期求值，与数组内容无关；
 - `sizeof(a+1)` 仍是指针大小（8）
 - `sizeof(*a)` 仍是第一行大小（12）

题解

这里要说出三个输出中详细的计算过程

分步推导：

```

1 // 第1行
2 *(p + 3) = 4 // p指向1, +3跳到4 (跨行了)
3 (*(q + 1) + 1) = 5 // q+1指向第1行, 解引用得{4,5,6}首地址, +1取5
4 结果 = 4 + 5 = 9
5
6 // 第2行
7 *(a[1] - 1) = 3 // a[1]指向4, -1指向3 (跨行往前)
8 *(a[0] + 4) = 5 // a[0]指向1, +4跳到5
9 结果 = 3 + 5 = 8
10
11 // 第四行
12 (int*)(a + 1)指向 4
13 (int*)(a + 0)指向 1
14 /// 两个指向同一个数组的同类型指针相减, 结果是它们之间相差多少个“元素” 1 到 4之间相差了3个元素, 结果为3
15
16 // 第3行
17 sizeof(a + 1) = 8 // a+1退化为指针
18 sizeof(*a) = 12 // *a是a[0], 3个int = 12字节
19 结果 = 8 + 12 = 20
20
21 // 第4行
22 (p + 2)[1] = 6 // p 是指针数组, p表示的是第一行的第一个元素 1
23 // (p + 2)[1]是p 先后移动 3 个int, 指向 4
24
25
26
27 (*(q + 1))[-1] = 3 // q+1指向第1行{4,5,6}, 解引用后指向4, [-1]取3 // q是数组指针, 指向第一个数组第一个元素
28 // (q + 1)指向第二个数组, (q + 1)[-1]指的是向前移动一位, 指向 3

```

29 | 结果 = 4 - 3 = 1

验证:

```
1 printf("%d\n", *(p + 3) + (*(q + 1) + 1)); // 4 + 5 = 9
2 printf("%d\n", *(a[1] - 1) + *(a[0] + 4)); // 3 + 5 = 8
3 printf("%d\n", (int)((int*)(a + 1) - (int*)(a + 0))) // 4 - 1 = 3
4 printf("%zu\n", sizeof(a + 1) + sizeof(*a)); // 8 + 12 = 20
5 printf("%d\n", (p + 2)[1] - (*(q + 1))[-1]); // 4 - 3 = 1
```